**Title: Three in One Student Services (TOSS)**

**Problem Statement/Rationale**

Organization is often recognized as a key component in a steady education, and slowly but surely, it lays the path for future success. With the hectic schedules and overflowing academic requirements, however, many scholars find it difficult to keep track of multiple other aspects of their studies (Orr, 2025). Disorderliness often inflicts stress and worry on a student, which adds once again to the importance of order.

With this in mind, the lack of said organization can be detrimental to a scholar's academic performance. For example, turning in homework late, which goes hand in hand with losing track of time, trouble with jotting down notes and prioritizing, cramming for exams last minute, and struggling overall with projects (Orr, 2026). These effects easily highlight the need for structure and order within a scholar's academic life, and even outside of it. While disorderliness can be easily undone, it's much better to target it before it starts.

**Project Overview and Relevance**

Three in One Student Services (TOSS) is a program designed to assist scholars in keeping their academic performance orderly and stable. The most notable features it has to offer are as follows; a requirement tracker that sorts them by importance, either by subject, deadline or type of requirement (FA, AA, ILA); a grade calculator for scholars to keep track of their marks and standing that uses user input of their raw grades and subject units; a course brochure that students can use as reference for their year/grade level's subjects; and precise error handling that validates user input.

As the world steps into a digital age, technology is being used more than ever, including in schools and general academics. Digital programs such as TOSS make assistance easily accessible for convenience. Having to perform multiple tasks such as the aforementioned ones in a physical manner can prove to be a hassle to some, which further reinforces the relevance of TOSS for the common problem of disorderliness that is commonly found in students.

**Objectives**

•General objectives

Develop a command-line application that assists students in managing academic tasks, including grade computation, requirement tracking, and course reference.

Specific Objectives

• Compute the General Weighted Average (GWA) based on user-input grades and subject credits.

• Convert raw percentage grades into a standardized grading scale using conditional logic.

• Collect and store subject-related data such as names, credits, and grades for processing.

• Track academic requirements by recording subject, type, and due date.

• Organize and prioritize tasks by sorting deadlines using the quicksort algorithm.

• Display requirements in order of urgency to help students manage time effectively.

• Provide a simple course brochure that lists subjects per grade level for student reference.

• Implement input validation and error handling to ensure reliable program execution

**Technical Approach**

The system used Python as its main programming language. It uses a menu system that contains different features for the users to select: Grade Calculator, Requirement Tracker, and Course Brochure.

The Grade Calculator allows the user to input the number of subjects, the names of the subjects, credits, and grade to calculate the General Weighted Average using a formula. The data analysis here processes these numbers to create a result that is submitted to the user.

The Requirement Tracker includes the CRUD operations. The data from the user is saved in a file and stored in the system.

Create - the user can add requirements to the list
Read - the program shows all the requirements added
Update - the user is given the ability to edit their requirements
Delete - the user can delete requirements if needed.

The Course Brochure prints/shows all courses offered and their respective course descriptions

**Methodology**

1. When the program is opened, the user will choose which feature they want to use for their specific requirement (1 for Grade Calculator, 2 for Requirement Tracker, 3 for Course Brochure, and 4 for exiting the program)

2. If the user chooses 1, the program asks the user how many subjects they have. The user will then input the number of subjects they are taking. If the user inputs a letter, a decimal number, or anything that's not an integer, the program will show an invalid input and will ask the user to press "Enter" to restart the process.

3. If it's valid, for how many times the user inputs the number of subjects, the program will ask for the subject name, the number of credits in that subject, and what the student's raw grade is for each subject.

4. The program will then calculate the General Weighted Average based on the data given by the user, and will return the final grade of the student. After that, the student can go back to the original page by pressing "Enter."

5. If the user chooses 2, they will be led to the guidelines on how to use the requirement tracker feature of the program. It will prompt the user to enter the subject, requirement type, and deadline. It also reminds the user to leave the space for an empty subject to finish the program. When inputting for the requirement type, if the user enters a different value than what the program gives, the program will show an invalid input.

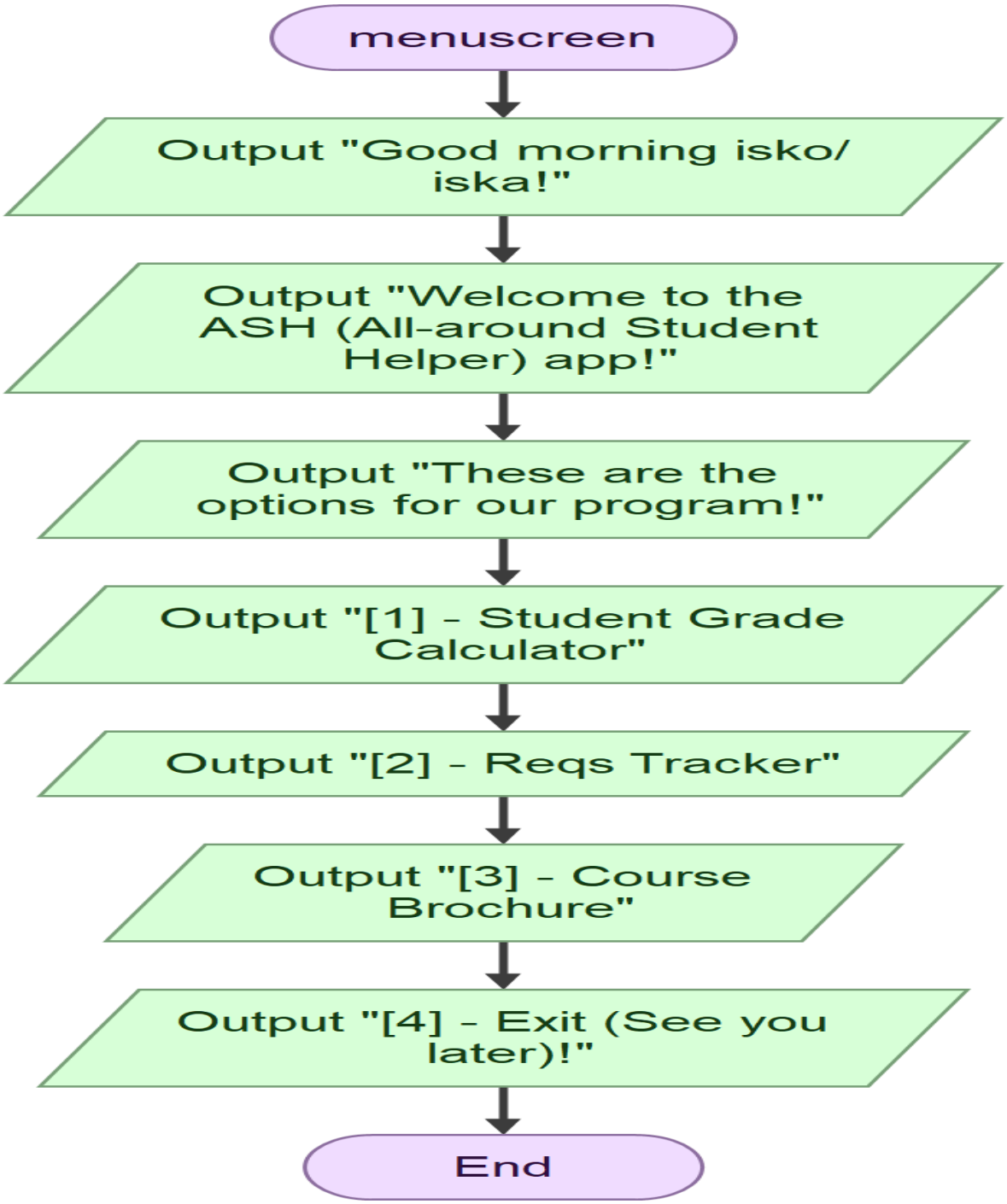
6. If input is valid, the program will ask the user to press enter to be led to the tracker itself. It will then ask for the subject, requirement type, and deadline over and over until the user leaves the subject empty. Once the user inputs an empty space, the program shows a list of all the requirements the user had previously inputted. The user can then go back to the original page by pressing "Enter".

7. If the user chooses 3, the program will output all courses offered and their respective course description. The program will also allow the user to go back to the program by pressing "Enter" once done.

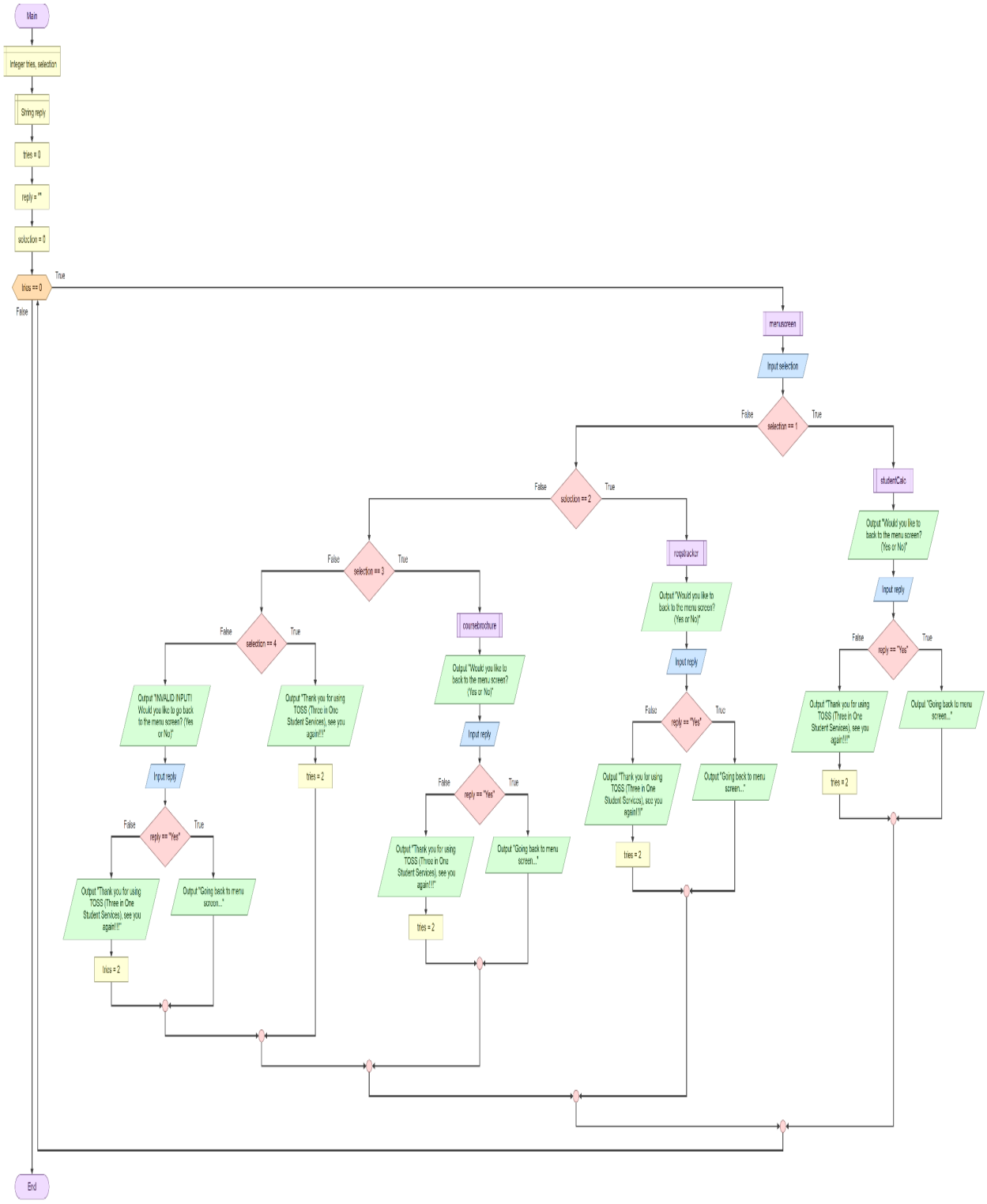
8. If the user chooses 4, the program will greet the user goodbye, and the user can press "Enter" to exit the program.

**Flowchart**

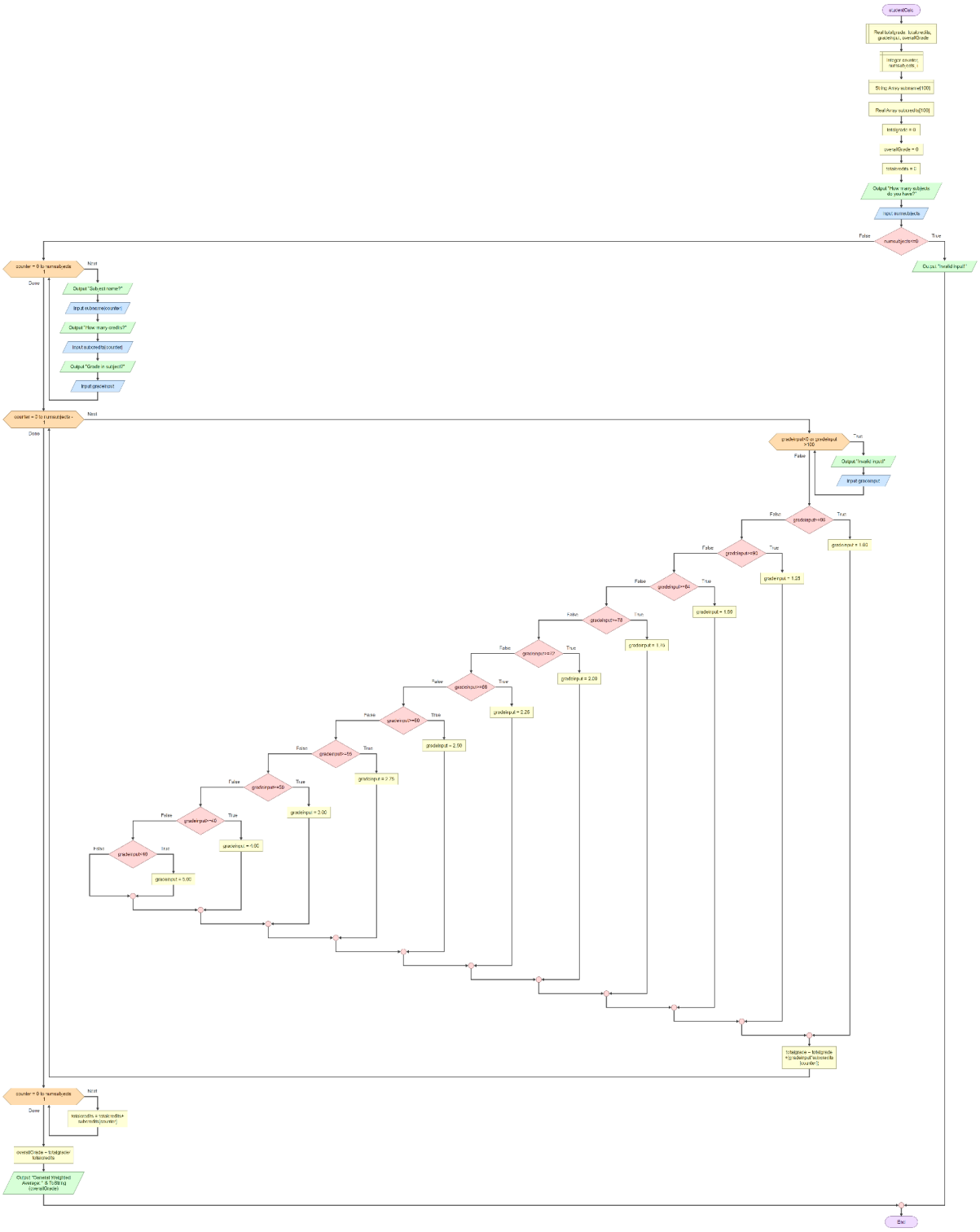
**Menu Screen**



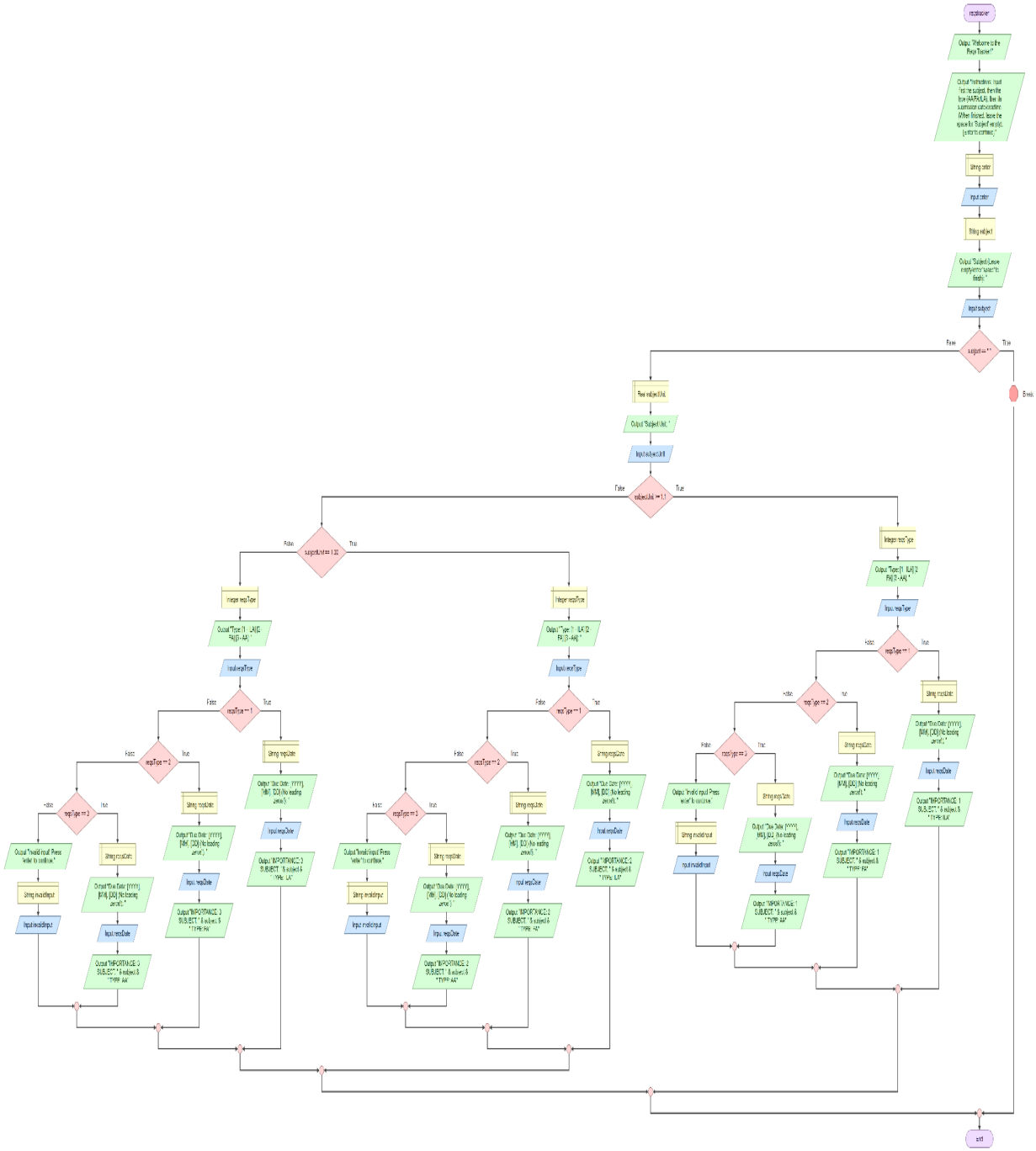
## Main Loop


Main
Integer tries, selection
String reply
tries = 0
reply = ""
selection = 0
tries == 0
True
False
menuscreen
Input selection
selection == 1
False
True
studentCalc
Output "Would you like to back to the menu screen? (Yes or No)"
Input reply
reply == "Yes"
Output "Thank you for using TOSS (Three in One Student Services), see you again!!!"
tries = 2
Output "Going back to menu screen..."
selection == 2
recapitulator
Output "Would you like to back to the menu screen? (Yes or No)"
Input reply
reply == "Yes"
Output "Thank you for using TOSS (Three in One Student Services), see you again!!!"
tries = 2
Output "Going back to menu screen..."
selection == 3
coursebrochure
Output "Would you like to back to the menu screen? (Yes or No)"
Input reply
reply == "Yes"
Output "Thank you for using TOSS (Three in One Student Services), see you again!!!"
tries = 2
Output "Going back to menu screen..."
selection == 4
Output "INVALID INPUT! Would you like to go back to the menu screen? (Yes or No)"
Input reply
reply == "Yes"
Output "Thank you for using TOSS (Three in One Student Services), see you again!!!"
tries = 2
Output "Going back to menu screen..."
Output "Thank you for using TOSS (Three in One Student Services), see you again!!!"
tries = 2
End

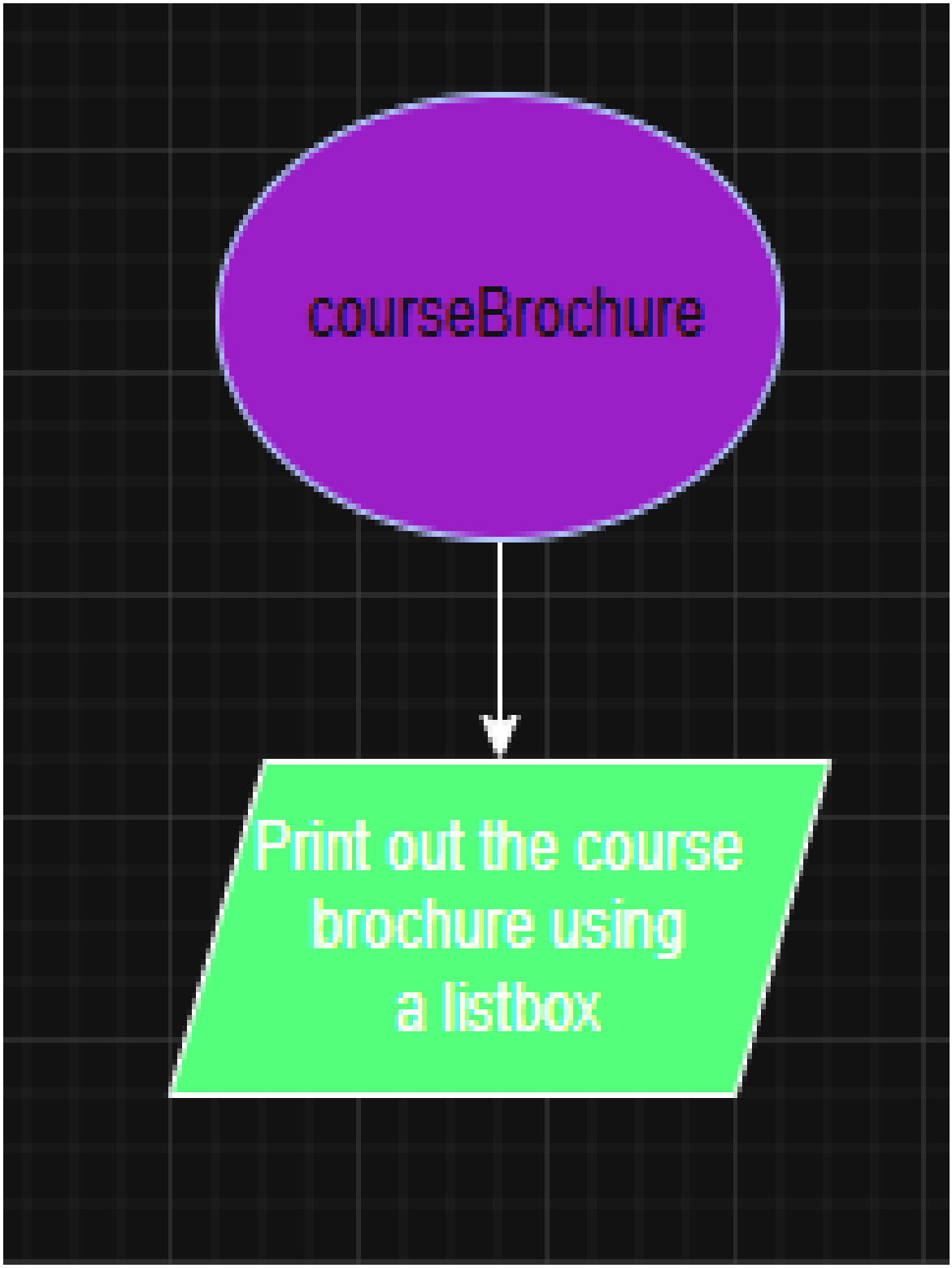
## Student Calculator/GWA Calculator



## Requirements Tracker

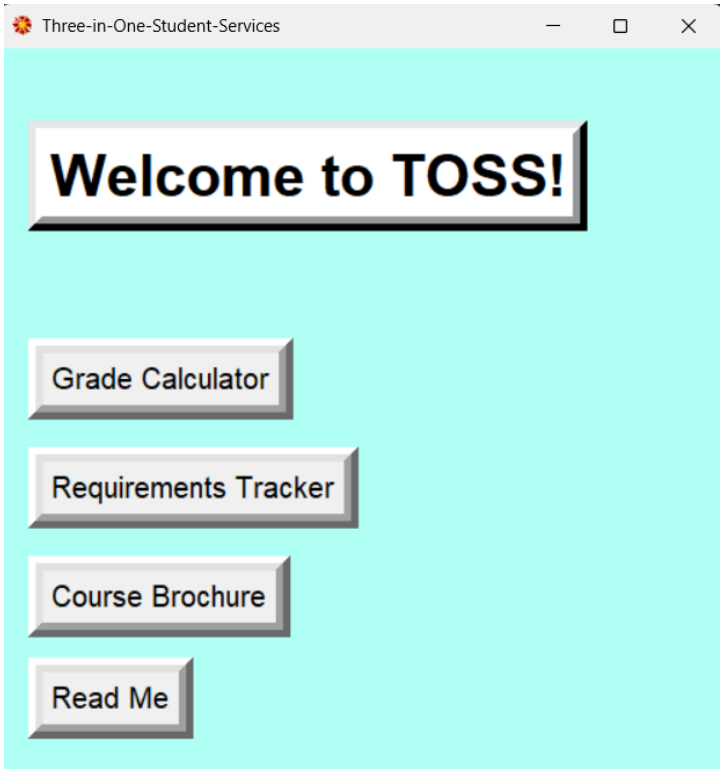


**Course Brochure**

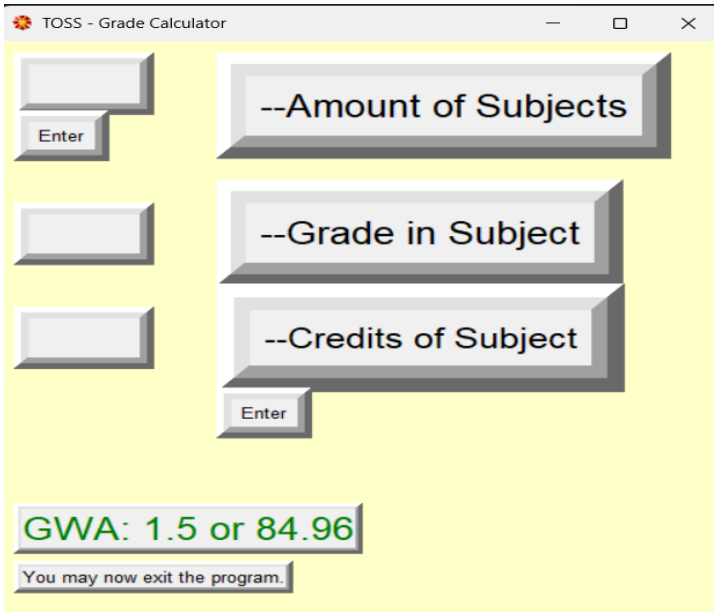


## Program

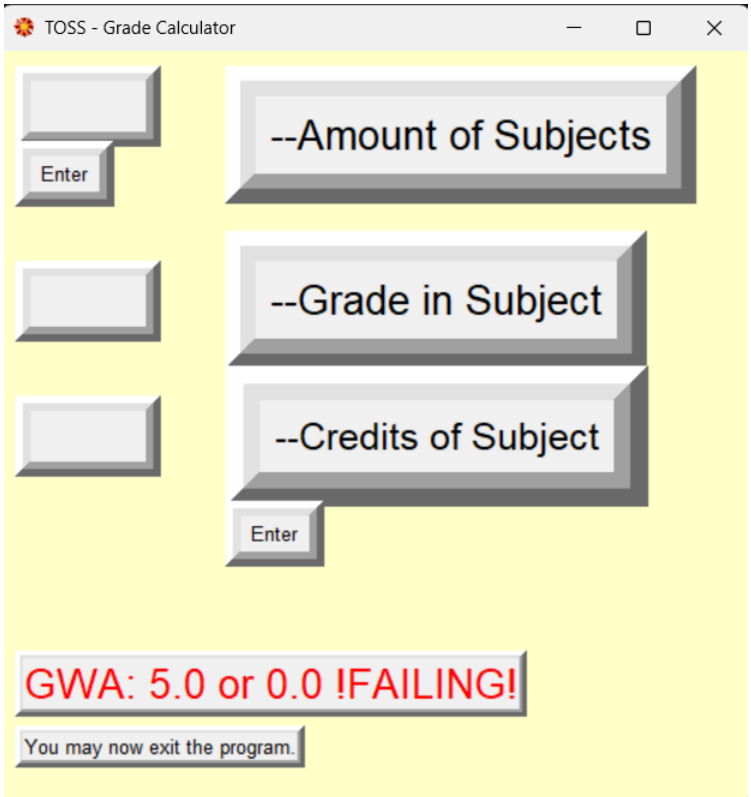
### Main Menu



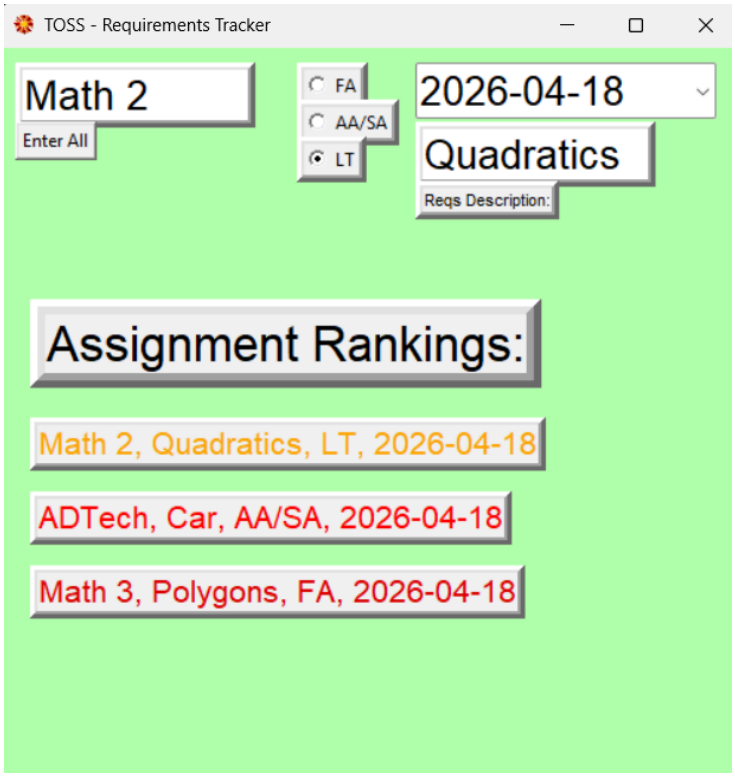

### GWA Calculator (If Proper Inputs)



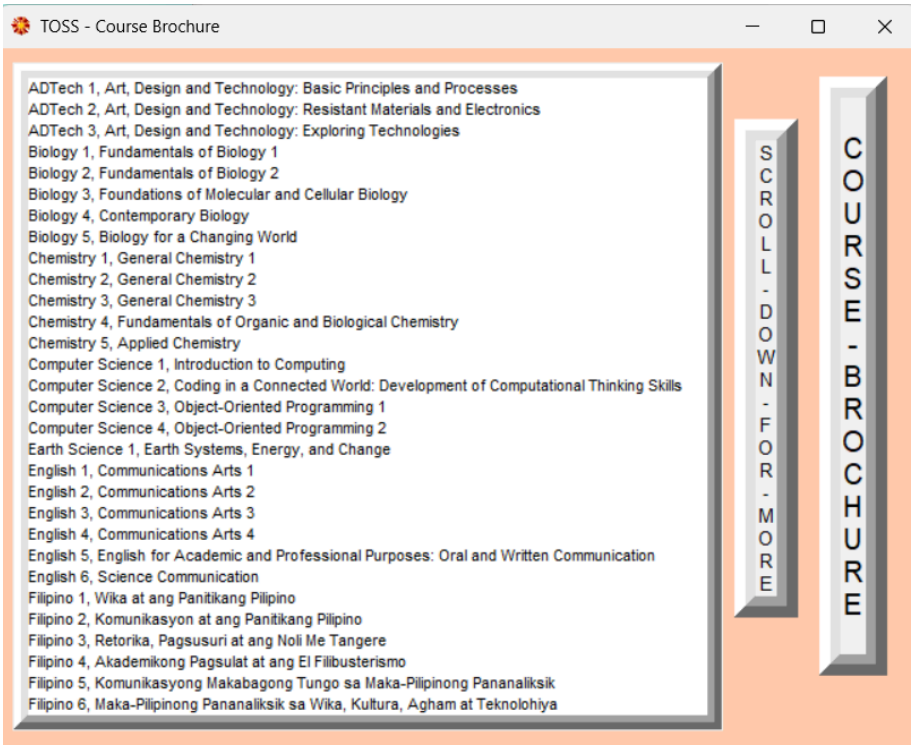
**GWA Calculator (If Not Proper Inputs)**



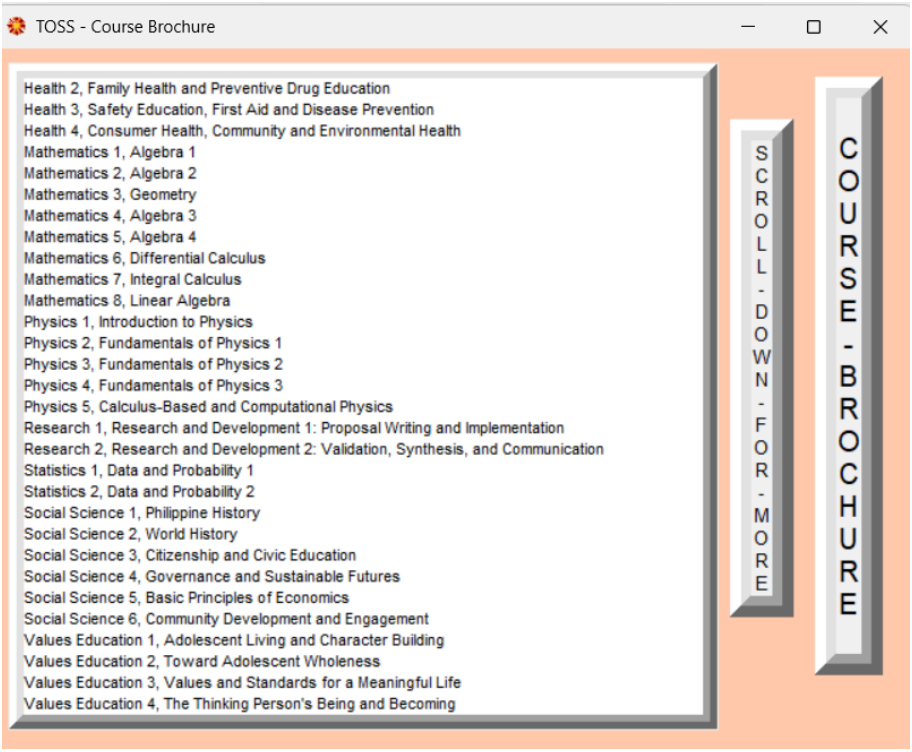

**Requirements Tracker**



## Course Brochure (Part 1)




## Course Brochure (Part 2)



**Team Roles**

| Team | Quarter 1 | Quarter 2 | Quarter 3 | Quarter 4 |
|---|---|---|---|---|
| Avilla | Part of the Paper (Problem Decomposition) | Main Programmer | Main Programmer | Main Programmer and README File |
| Castillo | Part of the Paper (Rest of the Paper) | Flowchart of Menu Screen and Course Brochure | Programmed Course Brochure | Part of the Final Paper (Team Roles) and PPT for Defense |
| Manalo | Part of the Paper (Scopes and Limitations) | Part of the Paper (Problem Decomposition) | Part of the README File (Features and Program Requirements) | Part of the Final Paper (Objectives and References) |
| Pagilagan | Tester and Feedback Giver | Flowchart of Requirement Tracker | Part of the README FILE (How to run program, Screenshots of output, Authors/Contributors | Part of the Final Paper (Flowchart and Methodology) |
| Yuson | Tester and Feedback Giver | Flowchart of Requirement Tracker | Part of the README File (Project Description and Objectives) | Part of the Final Paper (Problem Statement, Project Overview and Relevance) |

**References / Background Research**

Aho, A. V., Hopcroft, J. E., and Ullman, J. D. (1974). The design and analysis of computer algorithms. Addison Wesley. The Design and Analysis of Computer Algorithms (Aho, Hopcroft & Ullman 1974-01-11) PDF | PDF | Computational Complexity Theory | Time Complexity

Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. (2009). Introduction to algorithms. MIT Press. Introduction to Algorithms

Zimmerman, B. J. (2002). Becoming a Self-Regulated Learner: An Overview. Theory Into Practice, 41(2), 64–70. Becoming a Self-Regulated Learner: An Overview: Theory Into Practice: Vol 41, No 2

Orr, D. (2025, July 2). Why kids have trouble with organization. Oxford Learning. https://oxfordlearning.com/why-kids-have-trouble-with-organization/

Orr, D. (2026, February 20). How planning and organization skills boost your child's academic success. Oxford Learning. https://oxfordlearning.com/how-planning-and-organization-skills-boost-your-childs-academic-success/